

Contrastive Learning: InfoNCE & Cosine Similarity.

A RoBERTa encoder produces a vector per sentence — but raw vectors are not yet a *search index*. The InfoNCE objective pulls a query toward its true match and pushes it away from everything else, shaping a space where cosine similarity *means* relevance. Every number here — four cosine similarities, the temperature-scaled softmax, and the resulting loss — is computed in full, nothing summarised or deferred.

THE EQUATION

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(a, p)/\tau)}{\exp(\text{sim}(a, p)/\tau) + \sum_j \exp(\text{sim}(a, n_j)/\tau)}$$

Worked example. Anchor a and four candidates in $\mathbb{R}^3$ (one positive, three negatives); cosine similarities, temperature $\tau = 0.1$, softmax, and loss.

Topic 2 of 2 in this module · companion to the course page · v1.0

Contents

1	Cosine similarity: the relevance score	2
2	InfoNCE: a softmax over similarities	3
3	Worked example: one anchor, four candidates in $\mathbb{R}^3$	3
4	Properties / consequences that matter	4
5	Where this sits in the track	5
A	Reproducible (NumPy)	5

1 Cosine similarity: the relevance score

Embedding-based retrieval lives or dies on one scalar: how similar are two vectors? Magnitude is a distraction (a long movie synopsis should not out-score a short one merely for being long), so we compare *direction* via cosine similarity:

$$\text{sim}(u, v) = \cos \theta = \frac{u \cdot v}{\|u\| \|v\|} \quad (1)$$

It ranges in $[-1, 1]$: +1 identical direction, 0 orthogonal, -1 opposite. Equivalently it is the dot product of the *unit-normalised* vectors $\hat{u} = u/\|u\|$.

Symbol	Meaning	Shape
a	anchor / query embedding	vector in $\mathbb{R}^d$
p	positive (true match) embedding	vector in $\mathbb{R}^d$
n_j	negative (non-match) embeddings	vectors in $\mathbb{R}^d$
τ	temperature (> 0)	scalar
sim	cosine similarity	scalar in $[-1, 1]$
$\mathcal{L}$	InfoNCE loss for this anchor	scalar ≥ 0

Think of every sentence as an arrow on a sphere. Cosine similarity is just the angle between two arrows — it ignores how far out they sit. Contrastive training spins the arrows so that a query and its true match end up nearly parallel, while unrelated items splay off to the sides.

2 InfoNCE: a softmax over similarities

Given an anchor a , its positive p , and k negatives $n_1, \dots, n_k$, InfoNCE treats retrieval as a $(k+1)$ -way classification: “which candidate is the true match?” Each candidate gets a logit $\text{sim}(a, \cdot)/\tau$; a softmax turns the logits into probabilities; the loss is the negative log-probability assigned to the positive.

$$\mathcal{L} = -\log \frac{e^{\text{sim}(a,p)/\tau}}{e^{\text{sim}(a,p)/\tau} + \sum_{j=1}^k e^{\text{sim}(a,n_j)/\tau}} \quad (2)$$

Minimising $\mathcal{L}$ drives the positive’s similarity *up* and every negative’s similarity *down* — the “pull together, push apart” that gives contrastive learning its name.

The temperature τ . Dividing by τ sharpens (small τ) or softens (large τ) the softmax. Small τ (e.g. 0.05–0.1) makes the loss focus hard on the *closest* negatives — the confusable ones — producing crisp, well-separated embeddings. Too small and gradients explode on noise; too large and the model barely distinguishes positive from negatives.

3 Worked example: one anchor, four candidates in $\mathbb{R}^3$

Let the anchor and candidates be

$$a = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}, \quad p = \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix}, \quad n_1 = \begin{bmatrix} -1 \\ 2 \\ -2 \end{bmatrix}, \quad n_2 = \begin{bmatrix} 3 \\ -1 \\ 0 \end{bmatrix}, \quad n_3 = \begin{bmatrix} 0 \\ -2 \\ 1 \end{bmatrix}.$$

3.1 Step 1 — norms and dot products

$\|a\| = \sqrt{1+4+4} = 3$. Dot products with a and candidate norms:

$$\begin{aligned} a \cdot p &= 1 \cdot 2 + 2 \cdot 3 + 2 \cdot 4 = 16, & \|p\| &= \sqrt{4+9+16} = \sqrt{29} \approx 5.3852, \\ a \cdot n_1 &= -1 + 4 - 4 = -1, & \|n_1\| &= \sqrt{1+4+4} = 3, \\ a \cdot n_2 &= 3 - 2 + 0 = 1, & \|n_2\| &= \sqrt{9+1+0} = \sqrt{10} \approx 3.1623, \\ a \cdot n_3 &= 0 - 4 + 2 = -2, & \|n_3\| &= \sqrt{0+4+1} = \sqrt{5} \approx 2.2361. \end{aligned}$$

3.2 Step 2 — cosine similarities (Eq. 1)

$$\begin{aligned} \text{sim}(a, p) &= \frac{16}{3 \cdot 5.3852} = 0.99038, & \text{sim}(a, n_1) &= \frac{-1}{3 \cdot 3} = -0.11111, \\ \text{sim}(a, n_2) &= \frac{1}{3 \cdot 3.1623} = 0.10541, & \text{sim}(a, n_3) &= \frac{-2}{3 \cdot 2.2361} = -0.29814. \end{aligned}$$

The positive is already nearly aligned (0.990); the negatives sit near orthogonal or below. Heatmap of the similarity row (intensity = $\frac{\cos+1}{2} \times 80$, so $+1 \rightarrow \text{indigo!80}$):

	p	n_1	n_2	n_3
$\text{sim}(a, \cdot)$	0.990	−0.111	0.105	−0.298

3.3 Step 3 — divide by $\tau = 0.1$ (logits)

$$z = \frac{\text{sim}}{\tau} = 10 \cdot \text{sim} = [9.9038, -1.1111, 1.0541, -2.9814].$$

The $10\times$ amplification by $\tau = 0.1$ stretches the gap: a 0.99 vs 0.11 similarity gap becomes a logit gap of nearly 9.

3.4 Step 4 — softmax

Exponentiate and normalise:

$$e^z = [20005.28, 0.3292, 2.8694, 0.0507], \quad \sum e^z = 20008.53.$$

$$\text{softmax}(z) = \left[\underbrace{0.999838}_p, \underbrace{1.6 \times 10^{-5}}_{n_1}, \underbrace{1.43 \times 10^{-4}}_{n_2}, \underbrace{3 \times 10^{-6}}_{n_3} \right].$$

	p	n_1	n_2	n_3
softmax	0.99984	0.00002	0.00014	0.00000

3.5 Step 5 — the loss (Eq. 2)

$$\mathcal{L} = -\log(0.999838) = 1.624 \times 10^{-4} \approx 0.00016.$$

Equivalently, $\mathcal{L} = -z_p + \log \sum_j e^{z_j} = -9.9038 + \log(20008.53) = 0.000162$. The model is already very confident in the correct match, so the loss — and hence the gradient pressure — is tiny. A confusable negative (cosine near the positive's) would inflate $\mathcal{L}$ and produce a strong corrective gradient.

InfoNCE is a softmax-cross-entropy over similarity scores: the loss is small exactly when the positive's temperature-scaled similarity *dominates* every negative's. Minimising it sculpts an embedding space where cosine similarity is a calibrated relevance score — ready for nearest-neighbour retrieval.

4 Properties / consequences that matter

1. **Lower bound 0.** $\mathcal{L} \geq 0$, reaching 0 only as the positive's softmax probability $\rightarrow 1$. It can never be negative.
2. **Negatives are free supervision.** No labels needed beyond knowing one positive pair; all other in-batch items serve as negatives. A bigger batch $\Rightarrow$ more negatives per anchor $\Rightarrow$ a harder, more informative loss (ties this topic back to Topic 1's large batches).
3. **Temperature controls hardness.** Small τ concentrates the gradient on the hardest (closest) negatives; it is one of the most sensitive hyper-parameters in contrastive training.
4. **Scale invariance.** Cosine ignores vector magnitude, so the loss depends only on *angles* — exactly the quantity preserved by L2-normalising embeddings before indexing.

Forgetting to normalise. If you feed *raw dot products* into the softmax instead of *cosine* similarities, a long vector can win on magnitude alone, and the loss conflates “points the same way” with “is large”. Always divide by the norms (Eq. 1) — or equivalently L2-normalise both vectors first. Likewise, leaving the positive out of the denominator turns InfoNCE into a different (degenerate) objective: the positive term must appear in *both* numerator and denominator.

5 Where this sits in the track

A RoBERTa encoder gives a vector per sentence, but plain RoBERTa vectors are poor for similarity search. Sentence-BERT (SBERT) fine-tunes such an encoder with a contrastive / InfoNCE-style objective so that semantically related sentences land close in cosine space. That is precisely what powers *CineMatch* semantic search: encode the query and every film synopsis once, then retrieve by cosine similarity (approximate nearest neighbours). Topic 1 explained *how* to train the encoder at scale; this topic explains *what objective* turns it into a retrieval model.

A Reproducible (NumPy)

Inputs: a, p, n_1, n_2, n_3 as above, $\tau = 0.1$. The snippet regenerates every similarity, the softmax, and the loss.

```
import numpy as np
a = np.array([1.,2.,2.]); p = np.array([2.,3.,4.])
n1 = np.array([-1.,2.,-2.]); n2 = np.array([3.,-1.,0.]); n3 = np.array([0.,-2.,1.])
cos = lambda u,v: u@v/(np.linalg.norm(u)*np.linalg.norm(v))
sims = np.array([cos(a,p), cos(a,n1), cos(a,n2), cos(a,n3)])
tau = 0.1
z = sims/tau
soft = np.exp(z)/np.exp(z).sum()
loss = -np.log(soft[0])
print(np.round(sims,5))      # [ 0.99038 -0.11111  0.10541 -0.29814]
print(np.round(soft,6))     # [0.999838 0.000016 0.000143 0.000003]
print(loss)                  # 0.0001624081418...
```